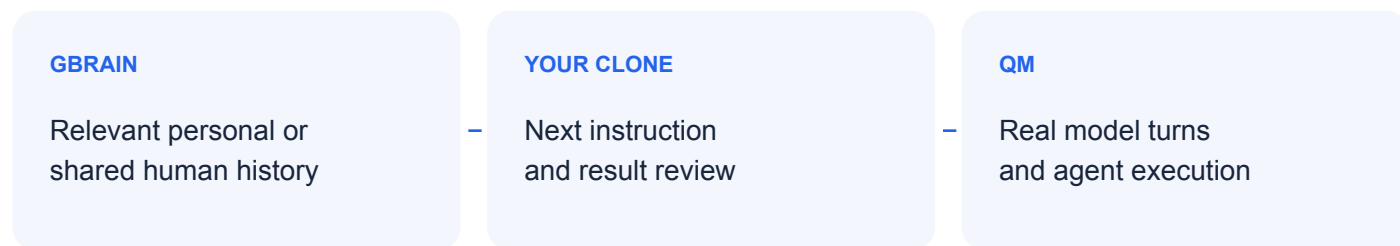


Your agent does the work. Your Clone decides what comes next.

A QM extension that turns remembered context into the next instruction, review, and correction.



Tab once: accept the suggested prompt.

Tab again: enable Clone. Stop: take control back.

WHAT THE DEMO PROVES

The review changes the work

The Clone caught a repeated call to action and requested tighter wording and specific sources. QM corrected the 50-word post and review checklist.

Memory has visible evidence

Official GBrain retrieves scoped human-history records for predictions and reviews. Evidence can be opened from the interface.

The result and Stop can be checked

We inspected the generated template and separately read back a paused session with no active run. The template still marks publication blocked until verification and approval.

[Working source + quickstart](#)

[93-second demo](#)

[Upstream QM PR](#)

The upstream PR is an open text proposal. The working implementation is in the fork.

One premise. One visible loop.

Approximate delivery targets. Pause at the handoff and let the correction speak.

30 SECONDS / 68 WORDS

Your agent does the work. You still supply every next decision.

Clone-in-the-Loop extends QM with a Clone grounded in your chat history. GBrain retrieves the relevant context. Your Clone predicts the next instruction and reviews the result.

Press Tab once to accept the suggested prompt. Press Tab again to hand over the loop. QM executes; your Clone requests corrections. Stop returns control to you.

Your judgment keeps working.

60 SECONDS / 145 WORDS

Agents can execute. Founders still write the follow-up, catch the missing detail, and start the next round.

We built Clone-in-the-Loop inside a QM source fork. Official GBrain retrieves relevant personal or shared chat history, with visible sources. Your Clone uses that context to propose your next instruction.

Press Tab once to accept it. Press Tab again and QM executes while your Clone reviews and directs the next step.

Watch this launch-review task. The Clone caught a repeated call to action, then requested tighter wording and specific sources. QM corrected the 50-word post and its review checklist. We inspected the template, stopped the loop, and checked that it stayed paused.

Switch to a team workspace to use shared judgment. Clone Garry is a fictional teammate with invented history; personal records stay outside team retrieval.

QM executes. GBrain remembers. Your Clone keeps your judgment in the loop.

The details behind the claim.

What did you build on QM?

A source extension with next-prompt prediction, continuous execution and review, scoped memory, teammate selection, and a compact web UI. QM supplies the actual turns and execution lifecycle.

How does GBrain change the result?

Its official PGLite engine stores human-history records. Source-scoped keyword recall supplies evidence for the next instruction and the review. The user can inspect those excerpts. Hosted federation and vector search are outside this build.

What did you verify?

Real reviews, corrections, and generated files; a paused state after Stop; restart persistence; and personal/team retrieval boundaries. A Product task produced a command-line tool whose three tests passed independently. See the linked verification record for exact boundaries.

Where does private context go?

Imported history is stored locally. Team retrieval excludes personal sources; selected excerpts go to the configured model provider. This is a trusted local prototype with one operator, not production multi-user isolation.

Is Clone Garry the real Garry?

Clone Garry is a fictional demo teammate with invented shared history. It demonstrates how a teammate's review preferences can guide the loop.

What is the upstream status?

The implementation is in the public QM fork. The open Upstream QM PR is a short text proposal under QM's contribution policy; it has not been merged or accepted.

PRIMARY LINKS

[Source + reproducible quickstart](#)

[Verification record](#)

[Official event + schedule](#)

[Upstream QM PR \(text proposal\)](#)